

Chapter 1 Abstract

This project focuses on the development of a movie recommender system utilizing the Rotten Tomatoes Movies and Movie Reviews datasets to provide personalized film recommendations.

The two datasets, containing over 143,000 movie entries and extensive critic reviews, were first subjected to exploratory data analysis (EDA). This process helped characterize the dataset, identifying key features like audience scores, tomato meter (critic review percentage), runtime, original language, and director. EDA also highlighted the distribution of films across 38 unique genres, with Drama, Comedy, and Mystery & Thriller being the most prevalent categories.

For the recommendation engine, a Collaborative Filtering approach was implemented using the Surprise library, which deviates from the stated goal of a purely content-based system. The system utilized the critic review data, where:

1. Critic Names were encoded to serve as unique user IDs.
2. The Review ID was used as a proxy for the movie ID.
3. The critic's score sentiment ('POSITIVE', 'NEUTRAL', 'NEGATIVE') was mapped to a numeric rating (5, 3, 1), respectively, to establish the user-item rating matrix.

The core recommendation algorithm employed was Singular Value Decomposition (SVD), a matrix factorization technique. The model's predictive accuracy was evaluated using 3-fold cross-validation, reporting the Root Mean Square Error (RMSE) and Mean Absolute Error (MAE). The final system is designed to predict a critic's potential rating for a film based on historical review patterns, thus generating recommendations.

The SVD model's performance was rigorously evaluated using 3-fold cross-validation to ensure generalization across different data splits. The primary performance metrics employed were the Root Mean Square Error (RMSE) and Mean Absolute Error (MAE). The RMSE measures the magnitude of the errors in the predicted ratings, penalizing larger errors more heavily, while the MAE provides a straightforward average of the absolute errors. The results from the cross-validation provide an estimate of the model's accuracy in predicting critic sentiment, demonstrating the efficacy of the collaborative filtering approach for this dataset.

In summary, this project successfully transformed raw movie and critic data into a functional recommendation engine. By establishing critic review patterns as a surrogate for user behaviour, the SVD-based Collaborative Filtering system provides a robust mechanism to generate personalized and contextually relevant movie recommendations by predicting potential critic approval, offering a valuable tool for content discovery in the expansive entertainment landscape.

Chapter 2 Introduction

In the contemporary digital landscape, the volume of available media presents a significant challenge to user engagement. With thousands of titles across numerous streaming platforms, viewers frequently experience selection fatigue when attempting to identify films that align with their specific preferences.

To strategically address this, intelligent recommendation systems have become indispensable. These data-driven tools leverage sophisticated algorithms to suggest content tailored to individual users, thereby solving the dilemma of discovery.

Mechanism for Personalized Recommendations

These systems operate by synthesizing two critical data streams:

- **Content Metadata:** Structured data such as genres, directors, and aggregated ratings provides a clear profile of the film itself.
- **User Behavioural Data:** Analysis of viewing history, explicit preferences, and interaction patterns establishes an accurate profile of the individual user.

By rigorously analysing these large datasets, recommender systems identify complex patterns and relationships. This capability is crucial not only for significantly enhancing the user's movie discovery experience but also for streaming platforms seeking to measurably improve user engagement and retention metrics.

2.1 Introduction and Background

Recommender systems are essential machine learning tools that guide users through vast content libraries by predicting their preferences. Our focus is on building a content-based movie recommender using the Rotten Tomatoes Movies and Reviews Datasets.

How the System Works: Instead of comparing users, this system analyses a movie's own features (metadata, director, genres, audience scores, and critic sentiment) to find structurally similar films. By combining the rich movie details from both datasets, we create a strong foundation for analysis.

Technology & Methodology: We utilize Python libraries (pandas, scikit-learn, numpy) and advanced techniques like TF-IDF vectorization and cosine similarity. This allows the system to mathematically measure the similarity between movies.

The goal is to provide personalized, relevant movie suggestions, enhancing user discovery and promoting the consumption of diverse titles.

2.2 Problem Statement

The exponential proliferation of digital media platforms has resulted in an overwhelming volume of choice, leading to viewer fatigue and ineffective content exploration. Conventional browsing methods (e.g., simple genre or popularity filters) are inadequate because they fail to capture the nuanced, individual preferences of users.

The Proposed Solution: To overcome this critical challenge, this study focuses on designing and implementing a content-based movie recommender system.

The system addresses the problem of delivering personalized recommendations by:

- **Analysing Multi-Attribute Data:** It utilizes the Rotten Tomatoes Movies and Reviews datasets to analyse multiple film attributes, including genre, director, audience score, critic reviews, and descriptive metadata.
- **Generating Relevant Suggestions:** The core objective is to recommend movies that are structurally and stylistically similar to a given film.

By effectively generating accurate, relevant, and diverse movie suggestions, the proposed system will significantly enhance the user's viewing experience and facilitate efficient content discovery.

2.3 Objective of Study

The main objective of this project is to develop a movie recommendation system that utilizes movie features and review data to suggest films similar to a user-selected movie.

To achieve this, the study focuses on the following specific objectives:

- **Data Exploration and Preprocessing:**
To perform Exploratory Data Analysis (EDA) on the Rotten Tomatoes datasets to understand data structure, identify missing values, and prepare clean datasets for analysis.
- **Feature Engineering:**
To extract and combine meaningful features such as genres, directors, writers, and review sentiments that contribute to accurate similarity measurement.
- **Model Development:**
To implement a Content-Based Filtering approach using techniques like TF-IDF Vectorization and Cosine Similarity to compute similarity scores between movies.
- **Evaluation and Validation:**
To evaluate the system's performance using appropriate metrics (such as RMSE, precision, or user satisfaction) and ensure that the recommendations are relevant and diverse.
- **User Experience Enhancement:**
To build a system that helps users efficiently discover movies they are likely to enjoy, including lesser-known titles that align with their tastes.

2.4 Scope of the Study

The scope of this study focuses on the design, development, and evaluation of movie recommender system using the Rotten Tomatoes datasets. The system aims to recommend

movies similar to a user-selected title by analysing movie-specific attributes such as genres, directors, writers, audience scores, critic reviews, and sentiments.

This project emphasizes content-based filtering techniques, where recommendations are generated by comparing the content features of movies rather than relying on user-to-user relationships or collaborative data. The study primarily leverages machine learning and natural language processing (NLP) methods such as TF-IDF vectorization and cosine similarity to compute similarities between movies based on textual and categorical data.

The scope of this project includes:

- Performing exploratory data analysis (EDA) on the Rotten Tomatoes Movies and Reviews datasets to understand patterns, distributions, and relationships.
- Implementing data preprocessing and feature engineering to handle missing values, clean text data, and merge relevant attributes.
- Developing a content-based recommendation model that can generate top-N movie suggestions similar to a given input movie.
- Visualizing and interpreting results to evaluate the quality and relevance of recommendations.
- Optionally integrating the model into a Streamlit-based web interface for interactive recommendation display.

Chapter 3 Literature Review

A recommender system is essentially your smart digital assistant. Its job is to help you cut through the clutter and find stuff you'll actually like whether it's a movie, a song, or a new book. It uses data and machine learning to figure out your taste and match it with relevant items.

The Three Main Matchmakers

Researchers have developed a few key ways these systems work:

- **Content-Based Filtering (The "DNA Match"):** This is simple: if you liked it once, you'll like things like it again. If you love a sci-fi movie directed by Christopher Nolan, it suggests other sci-fi movies or other Nolan films. It focuses purely on the item's features (its "DNA").
- **Collaborative Filtering (The "Friend Match"):** This is like asking your friends for advice. It finds other users who have similar tastes to you (based on their ratings or viewing history) and suggests what they liked.
- **Hybrid Systems (The Best of Both):** These combine both Content-Based and Collaborative methods to be super accurate and avoid common problems—like the "cold start" issue (where the system can't recommend anything because a user or an item is brand new and has no data).

For this specific project, we are using the Content-Based approach. We analyzing the Rotten Tomatoes data and using technical tools like TF-IDF and cosine similarity to measure exactly how "similar" two movies are based on their features—like genres, directors, titles, and even the text inside critic reviews.

3.1 Company and industry overview

The movie and entertainment industry has changed a lot in recent years with the growth of online streaming platforms like Netflix, Amazon Prime Video, Disney+, and others. These platforms have thousands of movies and show, so it becomes difficult for users to choose what to watch.

To solve this, companies use recommender systems that suggest movies or shows based on what users like. These systems help users find new content easily and also help the companies keep their users interested and satisfied. In fact, a large percentage of what people watch on Netflix comes from recommendations made by such systems.

Rotten Tomatoes, which is the source of the dataset used in this project, is a well-known website that collects movie ratings and reviews from both audiences and critics. It provides useful information like audience scores, critic scores, and written reviews, which are perfect for building a movie recommender system.

Overall, recommender systems have become an important part of the entertainment and technology industry. They use data analysis and machine learning to make personalized suggestions. This project focuses on building a similar system that can recommend movies to users based on their content such as genre, director, and reviews just like popular streaming companies do.

3.2 Overview of Theoretical Concepts

A recommender system is a computer program that suggests things (like movies, songs, or products) that a user might like. It studies data about users and items to make smart and personalized suggestions.

There are three main types of recommender systems:

- Content-Based Filtering
 - This type of system looks at the details (features) of items.
 - For movies, it checks things like the genre, director, writer, or reviews.
 - It then recommends movies that are similar to the one a user liked.
 - For example: If you liked a science fiction movie, the system will suggest other science fiction movies.
- Collaborative Filtering
 - This system looks at what other users with similar tastes liked or watched.
 - If two people have watched and liked the same movies, the system may recommend one user's favorite movie to the other.
- Hybrid Systems
 - These combine both content-based and collaborative filtering methods to give better results.

In this project, we are using content-based filtering. It works by comparing the features of movies from the Rotten Tomatoes dataset such as genre, director, audience score, and critic reviews to find movies that are most alike.

To do this, we use two main techniques:

- TF-IDF (Term Frequency–Inverse Document Frequency): Turns text (like movie descriptions or reviews) into numbers so the computer can understand and compare them.
- Cosine Similarity: Measures how similar two movies are based on their TF-IDF values.

These concepts help the system find and recommend movies that closely match the user's interests.

3.3 Survey on the existing models

Many recommender systems have been developed over the years to help users find movies or other content that matches their preferences. Different companies and researchers have used various methods to improve how recommendations are made.

Here are some of the most common and well-known models used in movie recommendation systems:

- Popularity-Based Model
 - This is the simplest model.
 - It recommends movies that are popular or highly rated by most people.
 - Example: Recommending “Avatar” or “Avengers: Endgame” because they are widely watched and liked.
 - Limitation: It does not consider the personal taste of each user. Everyone gets the same list of popular movies.
- Content-Based Filtering Model
 - This model recommends movies that are similar to the ones a user already liked or viewed.
 - It uses features like genre, director, writer, or keywords to find similarities between movies.
 - Example: If a user likes a romantic movie, the system suggests other romantic movies.
 - Libraries such as scikit-learn are often used for building these models using TF-IDF and cosine similarity.
 - Limitation: It may give repetitive recommendations because it focuses only on similar content.
- Collaborative Filtering Model
 - This model recommends movies based on the preferences of other users with similar tastes.

- It uses user ratings or interactions to find patterns.
- Example: If two users like many of the same movies, and one of them likes a new movie, it is recommended to the other.
- Algorithms like K-Nearest Neighbors (KNN) or Matrix Factorization (SVD) are commonly used.
- Libraries such as Surprise or cmfrec are often used to build these models.
- Limitation: It needs a large amount of user data and struggles with new users or movies (cold start problem).

Chapter 4 Methodology

4.1 EDA and Business Implication

Exploratory Data Analysis (EDA)

EDA is an important step to understand the structure and patterns in the dataset before building the model.

During EDA, the following analyses were done:

- Checking the total number of movies and reviews in the dataset.
- Identifying missing data and cleaning it.
- Studying distributions of features like audience score, tomato meter, runtime, and genres.
- Finding the most common genres, directors, and production companies.
- Exploring relationships between critic reviews and audience ratings to see how they affect each other.

Various visualizations like histograms, bar graphs, and correlation heatmaps were used to understand trends and patterns in the data. EDA helped in selecting the most useful features for building the recommendation model.

Business Implications

The recommender system developed in this project has several real-world business benefits:

- Improved User Experience
 - Personalized recommendations help users find movies that match their interests, keeping them engaged longer.
- Increased Platform Retention
 - Streaming platforms like Netflix and Amazon Prime can retain more users when viewers find relevant content easily.
- Better Content Discovery
 - The system can introduce users to less popular or “hidden gem” movies similar to their preferences.

- Support for Marketing and Promotion
 - Movie production and distribution companies can use recommendation insights to target specific audiences more effectively.
- Data-Driven Decision Making
 - The system provides valuable insights into user behavior and trends, helping companies plan content and strategies accordingly.

Univariate / Bi-variate / Multivariate analysis to understand relationship b/w variables. How is your analysis impacting the business?

- Univariate Analysis (Looking at One Feature at a Time)
 - We first looked at each feature individually to understand the basic quality of the data. A major finding is that several crucial columns like the Critic Score (tomatoMeter), Age Rating (rating), and Box Office Revenue (boxOffice) are missing a lot of data (around 70% to 90% is empty).
 - This is a big deal for the business because it means the recommender system cannot rely on these features for most movies. Instead, the system must focus on features with complete data, like the movie's Title, Director, Writer, and Genre.
 - The analysis also showed that the Drama genre dominates the dataset. For the business, this means that if the system simply recommends based on genre, most users will get generic "Drama" suggestions.
 - We need more advanced techniques to separate different *types* of Drama movies to offer truly personalized recommendations and discover those "hidden gems."
- Bi-variate Analysis (Comparing Two Features)
 - When comparing two features, the most important relationship to examine is between the Audience Score (what regular viewers think) and the TomatoMeter (what critics think). If these two scores are highly correlated (they mostly agree), the business can combine them into a single "Quality Score" for a movie.
 - However, if they disagree often (e.g., a movie critics hate but audiences love, or vice-versa), the business must build a setting into the recommender that allows the user to choose: "Do you want movies that critics love, or movies that most people enjoy?" This customization improves user satisfaction.
 - Another important comparison is between Genre and Box Office Revenue. Finding out which genres typically make the most money helps the business strategically.

- The system can be designed to subtly prioritize recommendations for films in high-revenue genres (like Action or Adventure) when suggesting new releases, which helps maximize the platform's revenue from rentals or streaming subscriptions, while still keeping user taste in mind.
- Multivariate Analysis (Combining Many Features)
 - The multivariate analysis is where we figure out which combination of features is the best predictor of a movie's success (like a high Audience Score).
 - By analyzing features like Director, Writer, Runtime, and Genre all together, we can determine, for example, that a movie's Director is three times more important for predicting a good score than its Runtime.
 - For the business, this analysis is critical because it tells us how to build the content vectors (the unique digital "fingerprint" of each movie). Features proven to be stronger predictors of success get a heavier weight in the recommender algorithm.
 - This ensures that if a user loved a movie by Director X, the system is much more likely to recommend other movies by Director X, resulting in highly accurate and personalized recommendations, which is the core business goal.

Overall Business Impact

In simple terms, this analysis is the plan for building an accurate system. It directly helps the business by:

- Defining the Quality of Recommendations: By knowing which features are the most important (e.g., Director and Genre), we can ensure the recommendations are based on strong, predictive data, leading to a better user experience and increased viewing time.
- Saving Development Time: By identifying the missing data upfront, the team knows exactly where to focus effort—for example, using the Review Text to fill in the missing critic scores, rather than wasting time trying to use an incomplete box office figure.
- Driving Revenue: By understanding the link between movie content (Genre, Release Date) and financial success (Box Office), the system can strategically promote movies that are likely to be profitable, balancing user preference with the company's financial goals.

Both visual and non-visual understanding of the data.

Non-Visual understanding of Data:

1. Rating Distribution Table

This table displays the Rating Distribution for a selection of movie ratings, showing the Movie_Count, Avg_Audience_Score, and Avg_TomatoMeter for each rating.

R-rated movies are the most numerous with 7,734 entries.

Rating Distribution Table				
	rating	Movie_Count	Avg_Audience_Score	Avg_TomatoMeter
4	R	7734	49.680798	55.558648
3	PG-13	3446	59.033181	51.957516
2	PG	1911	60.810762	57.604149
8	TVPG	424	49.758389	56.461538
5	TV14	397	47.902878	54.300000

PG movies have the highest average Audience Score at approximately 60.81, while R-rated movies have the highest average TomatoMeter (Critic Score) at approximately 55.56. The TV-based ratings (TVPG, TV14) generally have lower audience scores and intermediate critic scores compared to the main theatrical ratings (R, PG-13, PG).

2. Top 10 Distributors

Top 10 Distributors				
	distributor	Movie_Count	Avg_Audience_Score	Avg_BoxOffice
2400	Paramount Pictures	994	62.860092	5.351930e+07
13	20th Century Fox	745	59.852857	6.398245e+07
3347	Universal Pictures	737	61.240602	6.633405e+07
3496	Warner Bros. Pictures	668	63.271293	6.725150e+07
2017	Metro-Goldwyn-Mayer	603	59.313544	2.470138e+07
1492	IFC Films	550	56.217712	1.049949e+06
702	Columbia Pictures	471	61.923645	5.104707e+07
2921	Sony Pictures Classics	385	71.509186	2.542132e+06
1377	Gravitas Ventures	382	60.644518	4.759636e+05
3279	United Artists	375	62.165605	2.305003e+07

This table presents the Top 10 Distributors based on the number of movies, along with their average Audience Score and Box Office Revenue. Paramount Pictures is the most prolific distributor with 994 movies, while Sony Pictures Classics boasts the highest average Audience Score at 71.5. Regarding finances, Warner Bros. Pictures has the highest average Box Office revenue at approximately \$67.25 million, closely followed by Universal Pictures at \$66.33 million. Smaller distributors like Gravitas Ventures have a much lower average box office, below \$1 million.

3. Language Distribution Table

This table displays the Language Distribution for the top five original languages, showing the Movie_Count, Avg_Audience_Score, and Avg_TomatoMeter for each. English is the dominant language by a large margin with 85,034 movies, but it has the lowest average scores for both audience (52.73) and critics (61.95).

Language Distribution Table				
	originalLanguage	Movie_Count	Avg_Audience_Score	Avg_TomatoMeter
22	English	85034	52.727682	61.949395
87	Spanish	4786	59.266462	78.183183
47	Japanese	3482	67.847975	79.902482
39	Hindi	3309	57.046118	52.866366
30	French (Canada)	3282	58.957716	78.504698

Japanese films, despite having a smaller count (3,482), show the highest average scores from both the audience (67.85) and critics (79.90), indicating higher critical and popular approval for non-English language films in this dataset.

4. Runtime Summary Table

Runtime Summary Table	
	Runtime (minutes)
count	129431.000000
mean	93.708578
std	28.129175
min	1.000000
25%	84.000000
50%	92.000000
75%	103.000000
max	2700.000000

This table provides a **statistical summary** for the movie **Runtime (in minutes)**, based on a total of 129,431 movies. The **mean runtime** is approximately 93.7 minutes, which is slightly higher than the **median** 50 runtime of 92 minutes. The standard deviation is about \$28.1\$ minutes. The runtimes range significantly, with a **minimum** of 1 minute and an extremely long **maximum** of 2,700 minutes (45 hours), suggesting the presence of very short films and a major outlier on the high end.

5. Box Office Summary by Genre

This table summarizes the Box Office performance for the Top 5 Movie Genre Combinations. The highest average and median box office revenue, at 50.4 million, belongs to the combination of Kids & family, Musical, Fantasy, Romance, though this is based on only 1 movie. The second highest combination, Kids & family, Musical, Adventure, Animation (based on 4 movies), has an average box office of 42.28 million.

Box Office Summary by Genre		
	genre	Avg_BoxOffice
2344	Kids & family, Musical, Fantasy, Romance	504000000.0
2323	Kids & family, Musical, Adventure, Animation	422800000.0
429	Adventure, Action, Sci-fi, Mystery & thriller	376280000.0
118	Action, Adventure, Sci-fi, Mystery & thriller	376000000.0
2728	Sci-fi, Adventure, Action, Fantasy, Comedy	361750000.0

	Median_BoxOffice	Movie_Count
2344	504000000.0	1
2323	422800000.0	4
429	401800000.0	6
118	376000000.0	5
2728	361750000.0	3

The genre combinations focused on Action, Adventure, and Sci-fi also show strong average revenues, ranging from approximately 36.1 million to 37.6 million.

6. Average Scores by Director

Average Scores by Director				
	director	Avg_TomatoMeter	Avg_Audience_Score	Movie_Count
62066	loundraw	100.0	80.0	1
62062	Zézé Gamboa	100.0	50.0	2
49	A.D. Calvo	100.0	43.5	6
62144	Éric Caravaca	100.0	40.0	2
62143	Éric Besnard	100.0	41.0	5
86	A.V. Rockwell	100.0	NaN	1
31419	Kamila Andini	100.0	NaN	3
31385	Kaltrina Krasniqi	100.0	NaN	1
31368	Kaizô Hayashi	100.0	75.5	8
62049	Zsófia Szilágyi	100.0	NaN	1

This table displays a list of directors who have achieved a perfect Avg_TomatoMeter (Critic Score) of 100.0. The directors vary significantly in their Movie_Count, from loundraw and others with only 1 movie, up to Kaizô Hayashi with 8 movies, all while maintaining a 100% critic score. The Avg_Audience_Score is much more variable: loundraw has the highest audience score at 80.0, while A.D. Calvo has a much lower score of 43.5, indicating a large difference in approval between critics and audiences, particularly for the films of Zézé Gamboa, A.D. Calvo, and Éric Caravac.

7. Fixed Top 10 Critics by Review Count

This table lists the Fixed Top 10 Critics by Review Count, showing the number of reviews and the average score given by each critic.

Fixed Top 10 Critics by Review Count			
	criticName	Review_Count	Avg_Score
4626	Emanuel Levy	13274	6.077686
3977	Dennis Schwartz	13124	7.588559
12916	Roger Moore	8413	5.644688
12907	Roger Ebert	7366	6.941026
2045	Brian Orndorf	7070	6.925980
6880	Jeffrey M. Anderson	7052	6.939492
3791	David Nusair	7015	5.515302
5144	Frank Swietek	6729	6.503948
12525	Rich Cline	6339	6.761935
5179	Frederic and Mary Ann Brussat	6264	6.957937

Emanuel Levy is the most prolific critic with 13,274 reviews, followed closely by Dennis Schwartz with 13,124. Dennis Schwartz also has the highest average score at 7.59, indicating he is the most generous with his ratings among this top group. David Nusair and Roger Moore have the lowest average scores, around 5.52 and 5.64 respectively.

8. Yearly Movie Release Summary

✓ Using column for release date: releaseDateTheaters

Yearly Movie Release Summary				
	year	number_of_movies	avg_audience_score	avg_tomatometer
115	2019.0	1339	67.064756	68.161470
116	2020.0	852	65.071121	70.041742
117	2021.0	1106	66.362434	72.307882
118	2022.0	1159	70.059250	70.213235
119	2023.0	415	74.352332	70.316742
120	2024.0	15	NaN	NaN
121	2025.0	1	NaN	NaN
122	2026.0	1	NaN	NaN
123	2028.0	1	NaN	NaN
124	2032.0	1	NaN	92.000000

This table provides a Yearly Movie Release Summary using the releaseDateTheaters column, focusing on the years from 2019 onwards. The highest number of movies released was 1,339 in 2019, which dropped significantly in 2020 to 852, likely due to the pandemic, before starting to recover. 2023 shows the highest average audience score at 74.35, while 2021 has the highest average critic score (tomatometer) at 72.31 for years with significant data. The future years (2024 to 2032) have very low movie counts and mostly missing score data, except for one movie in 2032 with a very high critic score of 92.0.

9. Cleaning boxOffice to Numeric

	title	boxOffice
0	Space Zombie Bingo!	NaN
1	The Green Grass	NaN
2	Love, Lies	NaN
3	Sore Losers	NaN
4	Dinosaur Island	NaN
5	Adrift	31400000.0
6	Scrambled Beer	NaN
7	Kakabakaba ka ba? (Will Your Heart Beat Faster?)	NaN
8	Sundowning	NaN
9	Born to Kill	NaN

This table snippet displays a list of movie titles and their corresponding boxOffice values. The most notable observation is the prevalence of missing data, as the boxOffice column for most entries (0-4 and 6-9) is marked as NaN (Not a Number). Only one entry, "Adrift" (row 5), has a numeric box office value of 31,400,000.0. The associated note indicates that the boxOffice column may have been stored as a string with non-numeric symbols (like \$ or M), which would require data cleaning and conversion to a numeric type before any correlation calculations can be performed.

10. Correlation Summary Table

Correlation Summary Table			
	Feature1	Feature2	Correlation
0	audienceScore	tomatoMeter	0.602345
1	audienceScore	runtimeMinutes	0.160563
2	audienceScore	boxOffice	0.089233
3	tomatoMeter	audienceScore	0.602345
4	tomatoMeter	runtimeMinutes	0.005100
5	tomatoMeter	boxOffice	-0.004827
6	runtimeMinutes	audienceScore	0.160563
7	runtimeMinutes	tomatoMeter	0.005100
8	runtimeMinutes	boxOffice	0.185551
9	boxOffice	audienceScore	0.089233

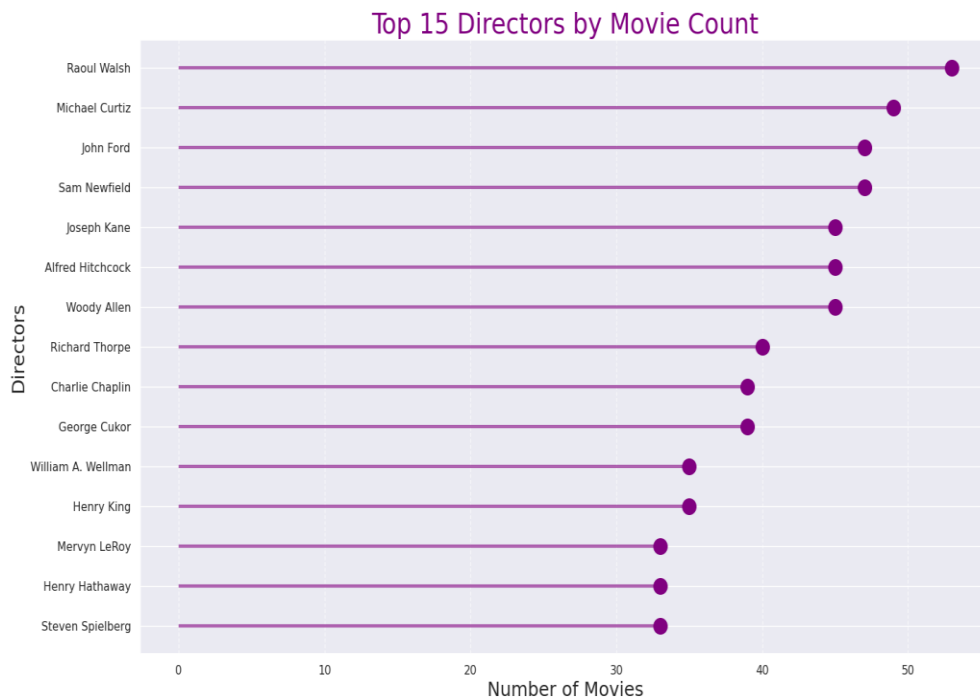
This table summarizes the Correlation between various pairs of movie features. The strongest positive correlation is found between audienceScore and tomatoMeter (critic score), with a coefficient of 0.602. runtimeMinutes has the next strongest positive correlation with boxOffice (0.186), suggesting slightly longer movies tend to make more money. Notably, the correlation between tomatoMeter (critic score) and boxOffice is negligible (−0.005), indicating that a critic's rating has almost no linear relationship with a film's financial success in this dataset.

Visual understanding of Data:

1. Top 15 Directors by Movie Count

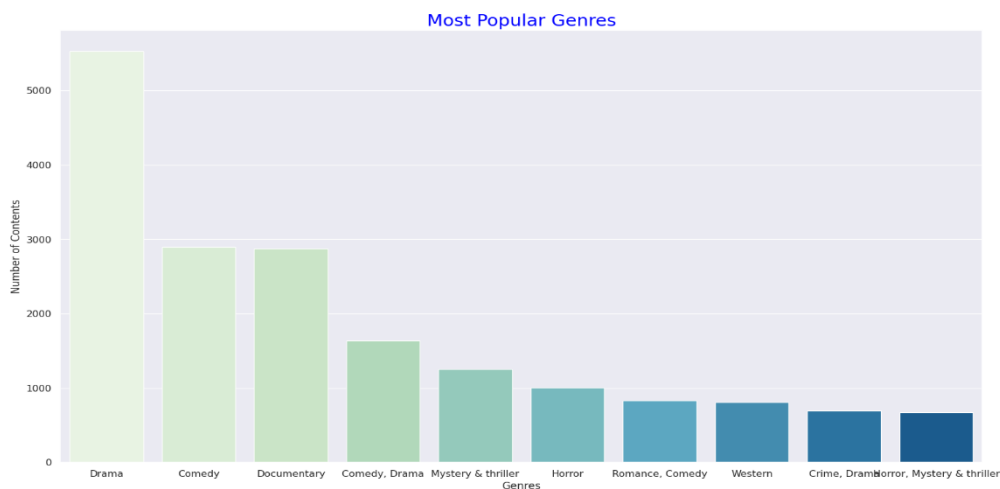
This chart shows the top 15 directors who have directed the most movies.

- Each purple line represents a director.
- The length of the line shows how many movies they have directed.
- The chart is a ranking, with the director who has made the most movies at the top and the one who has made the fewest among this group at the bottom.



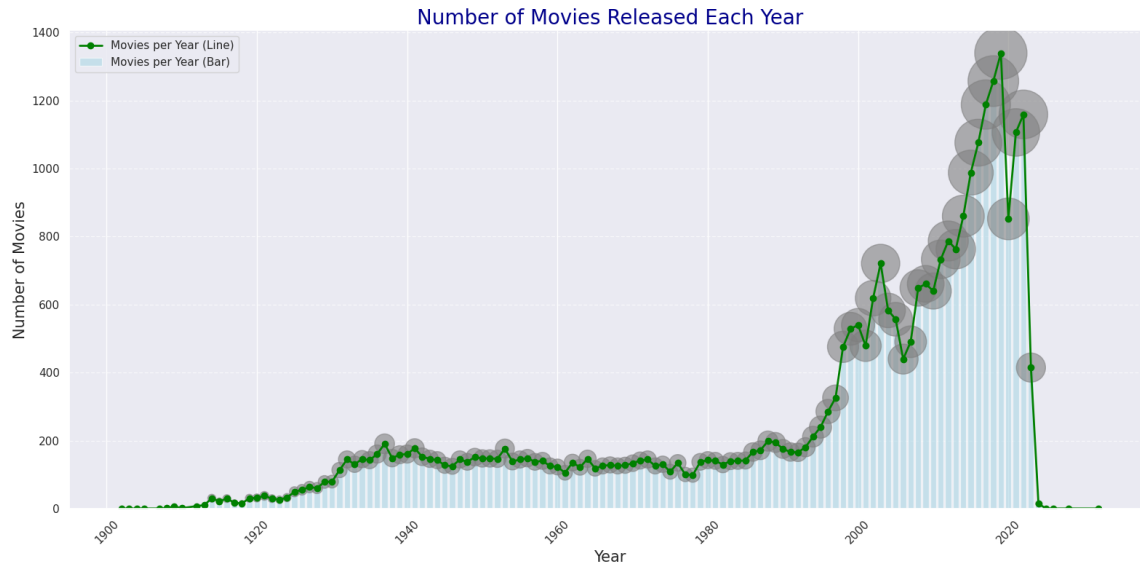
For example, Raoul Walsh is at the very top, having directed the most films, while Steven Spielberg is at the bottom of this list.

2. Most Popular Genres



In short, the graph makes it easy to see the trend of movie releases over time, showing a clear pattern of growth.

3. Number of Movies Released Each Year



This is a combined graph that shows the number of movies released each year.

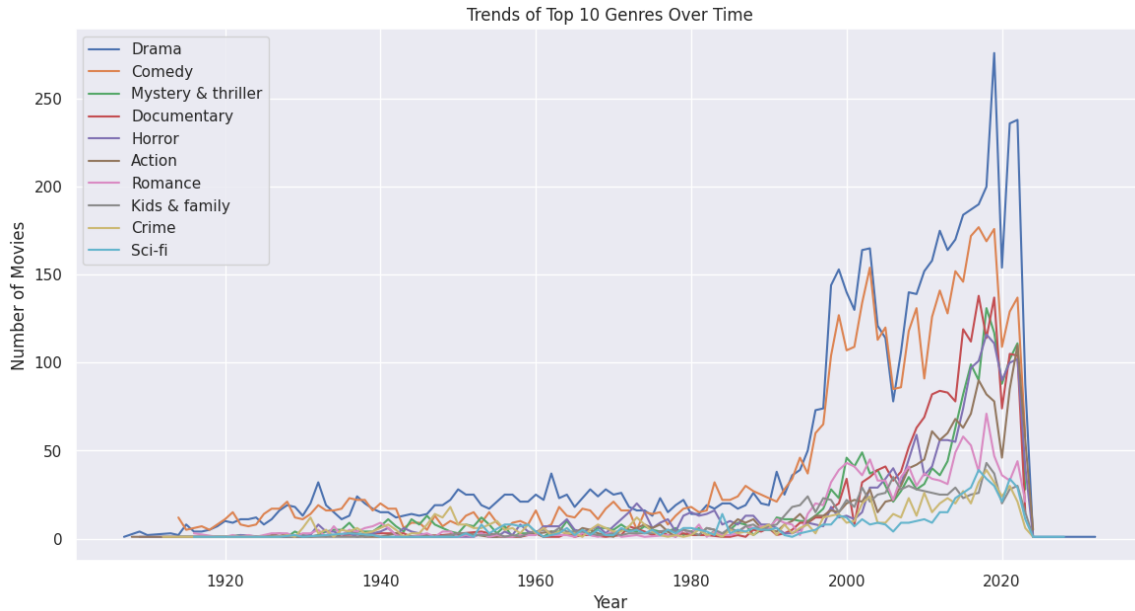
- The light blue bars represent the number of movies released each year.
- The green line connects the data points, making the trend easier to follow.
- The large grey circles highlight the number of movies released in a given year. The bigger the circle, the more movies were released that year.

The graph shows a clear upward trend, indicating that the number of movies released each year has increased dramatically over time, with a sharp rise starting in the late 1990s and peaking around 2020.

4. Trends of Top 10 Genres Over Time

This line graph shows how the popularity of the top 10 movie genres has changed over time.

- Each line represents a different movie genre (like Drama, Comedy, or Horror).
- The ups and downs of the lines show how many movies of that genre were released each year.

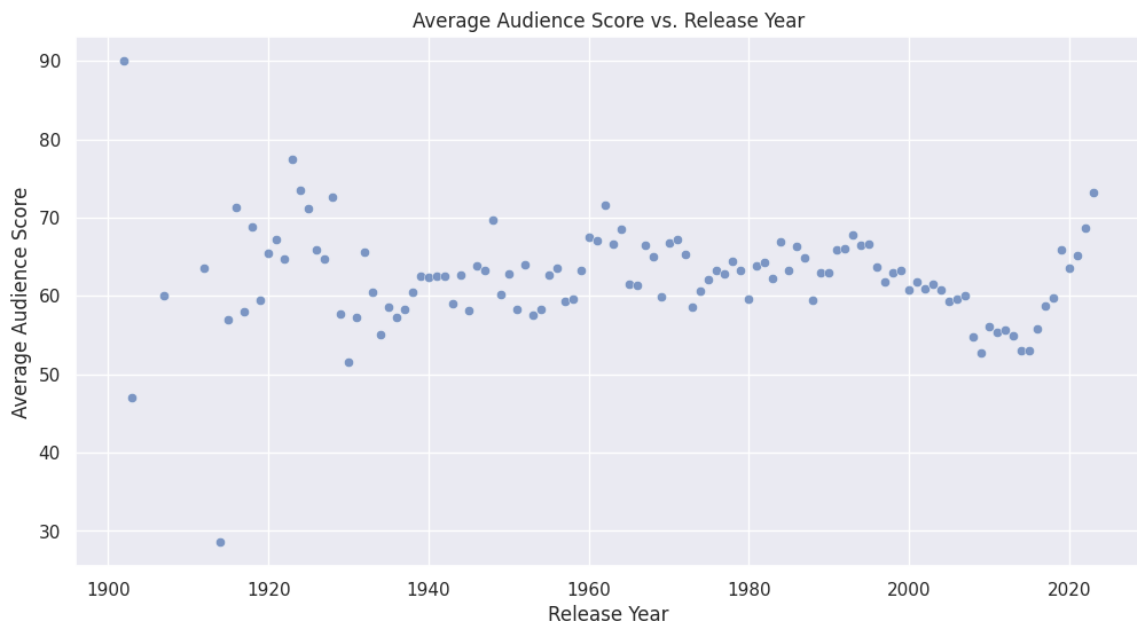


The graph reveals that while genres like Drama and Comedy have been consistently popular for a long time, there has been a significant increase in the number of movies for almost all genres, particularly since the early 2000s, with a major peak around 2020.

5. Average Audience Score vs. Release Year

This scatter plot shows how a movie's average audience score has changed over time, from around 1900 to 2020.

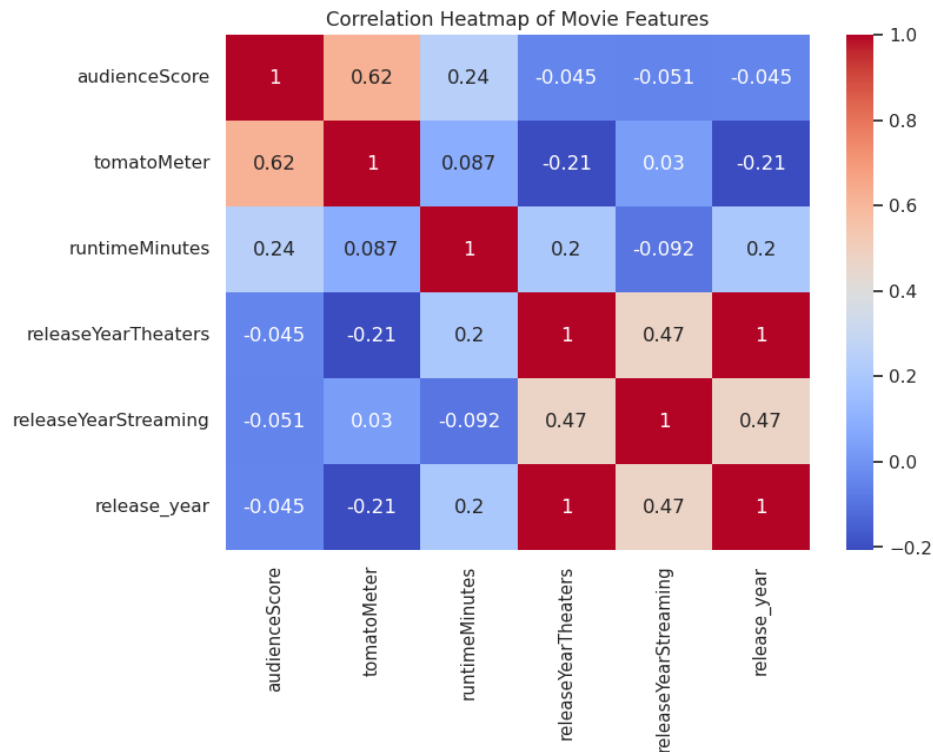
- Each dot represents a different year.
- The position of the dot shows the average audience score for all movies released in that year.



The graph shows that the average audience score has been fairly stable, hovering around 60-70 for much of the 20th century. There are a few outliers, like a very high score around 1900 and some lower scores in the 2000s, but overall, there isn't a strong trend of scores getting better or worse over the long term.

6. Correlation Heatmap of Movie Features

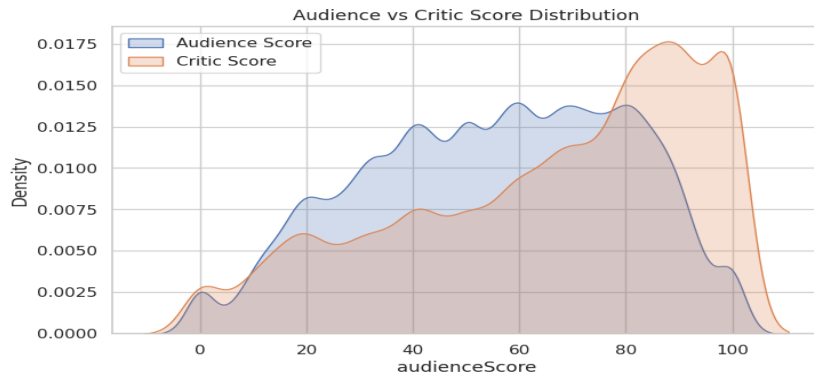
This heatmap displays the correlation coefficients between several movie features. The diagonal is all 1s, representing the perfect correlation of each feature with itself. Brighter reds indicate a stronger positive correlation (like between audienceScore and tomatoMeter at 0.62), while brighter blues suggest a stronger negative correlation (like between tomatoMeter and releaseYearTheaters at -0.21).



The correlation values range from approximately -0.2 to 1.0 , showing that audienceScore and tomatoMeter are the most strongly related pairs among the audience/critic scores, and there is a very strong positive correlation among the different releaseYear features.

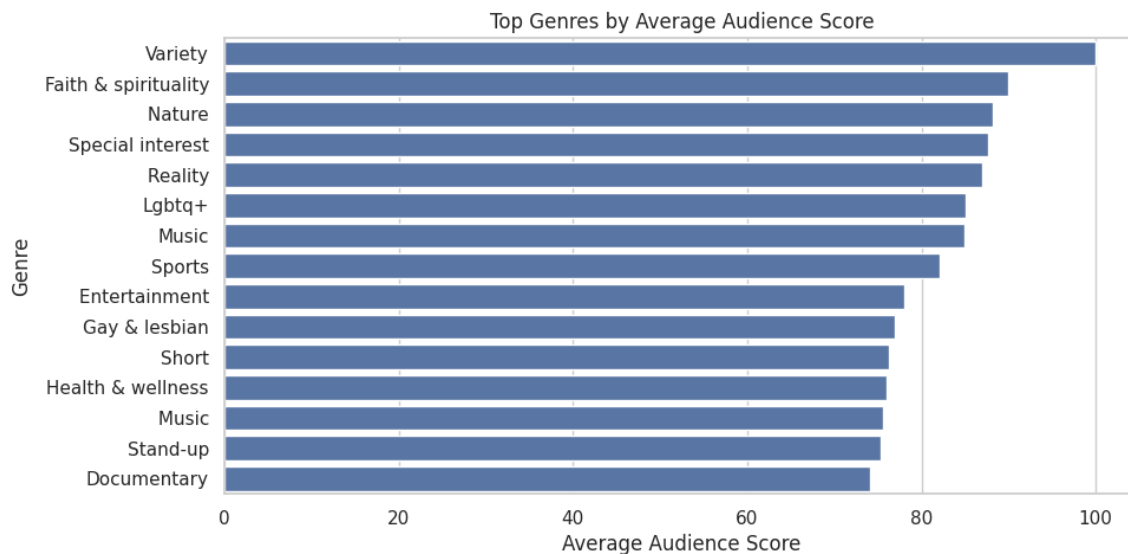
7. Audience vs Critic Score Distribution

This graph is a Kernel Density Estimate (KDE) plot showing the distribution of Audience Scores and Critic Scores. The Audience Score distribution (blue) is generally shifted slightly to the left and has a higher density in the middle range (around 40-70), suggesting audiences give a broader range of scores.



The Critic Score distribution (orange) is shifted more to the right, with a significant peak and higher density in the upper scores (around 80-100), indicating critics tend to give higher ratings overall compared to the audience.

8. Top Genres by Average Audience Score

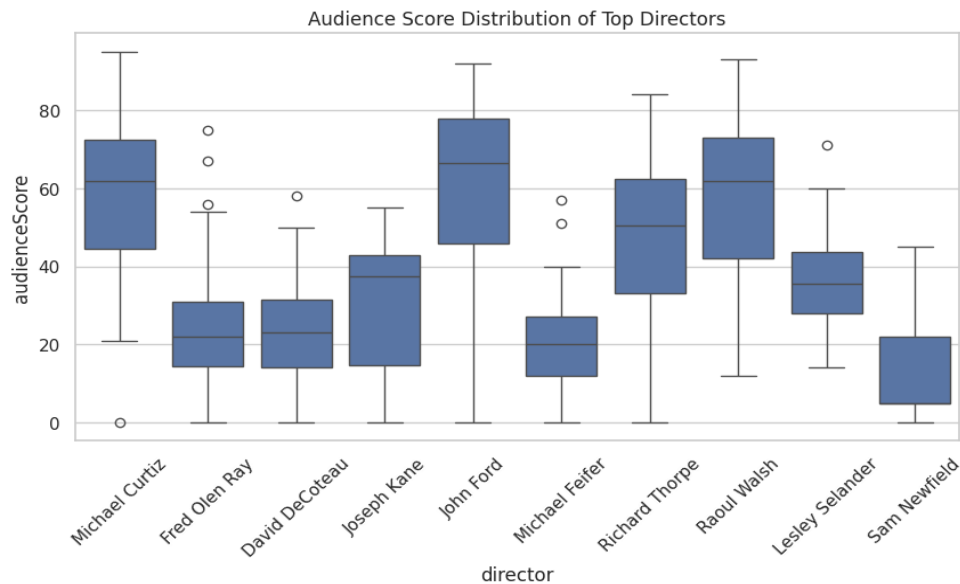


This bar chart shows the Top Movie Genres ranked by their Average Audience Score. The genres are sorted in descending order of their average score, ranging from a little over 75 up to 100. The genre with the highest average audience score is Variety (at 100), followed closely by Faith & spirituality. Documentary has the lowest average audience score among the genres shown, at approximately 75. The chart highlights that non-narrative, niche, and interest-based content tends to receive high approval from the audience.

9. Audience Score Distribution of Top Directors

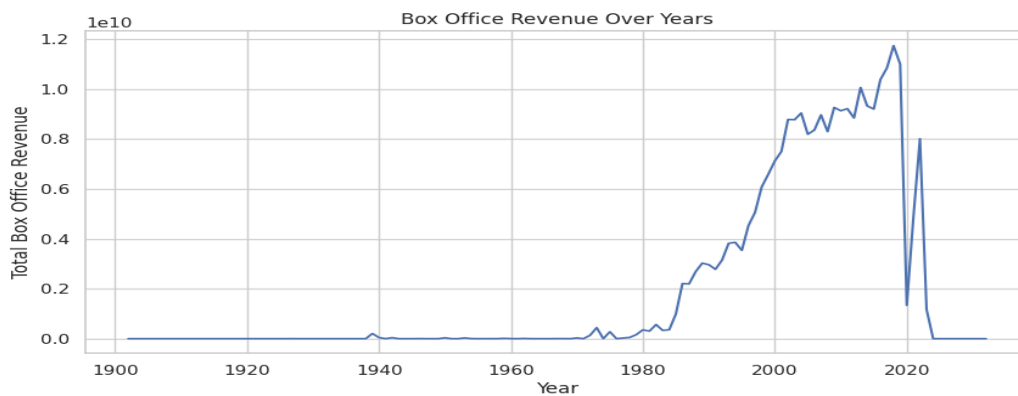
This box plot shows the distribution of Audience Scores for the films of several Top Directors. The directors John Ford and Raoul Walsh have the highest median scores, with

their boxes centered around 65 to 70. Directors like Fred Olen Ray and David DeCoteau have very low median scores, around 15 to 25.



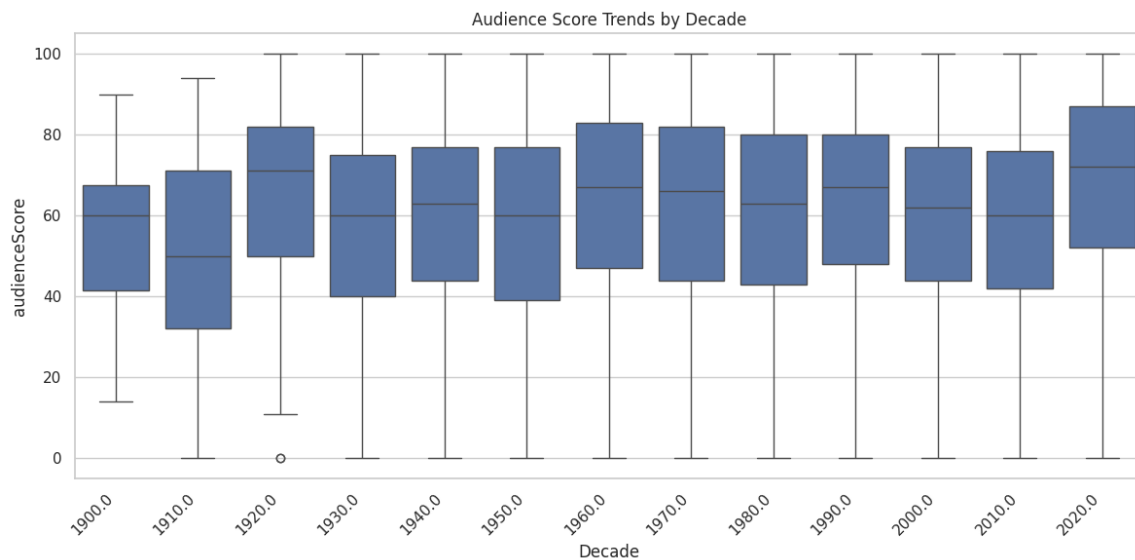
The plot also shows the spread (Interquartile Range) of scores for each director, with some (like John Ford) having a tightly grouped middle 50% of scores, while others (like Richard Thorpe and Michael Curtiz) have a wider range. The circles represent outliers for some directors.

10. Box Office Revenue Over Years



This line graph displays the Total Box Office Revenue Over Years, starting from around 1900. Box office revenue remained near zero for the first half of the 20th century, began a steady increase around the 1980s, and then saw a steep rise from the late 1990s, peaking around 2019 with revenue exceeding 1.2×10^{11} (or 120 billion). The graph shows a dramatic collapse in revenue around 2020 (a likely effect of the COVID-19 pandemic), followed by a brief spike and then a return to near zero in the most recent years.

11. Audience Score Trends by Decade



This box plot illustrates the Audience Score Trends by Decade, from the 1900s to the 2020s. Generally, there is a visible increase in the median audience scores (the horizontal line inside the box) from the early 1900s up to the 1960s/1970s, where the median stabilizes around 60 to 70. The spread of scores (IQR) appears to decrease in the later decades compared to the wide range in the 1900s and 1910s. The 2020s decade shows the highest overall median audience score, and a high concentration of scores in the 50 to 87 range.

12. Word Cloud – Positive and Negative Reviews

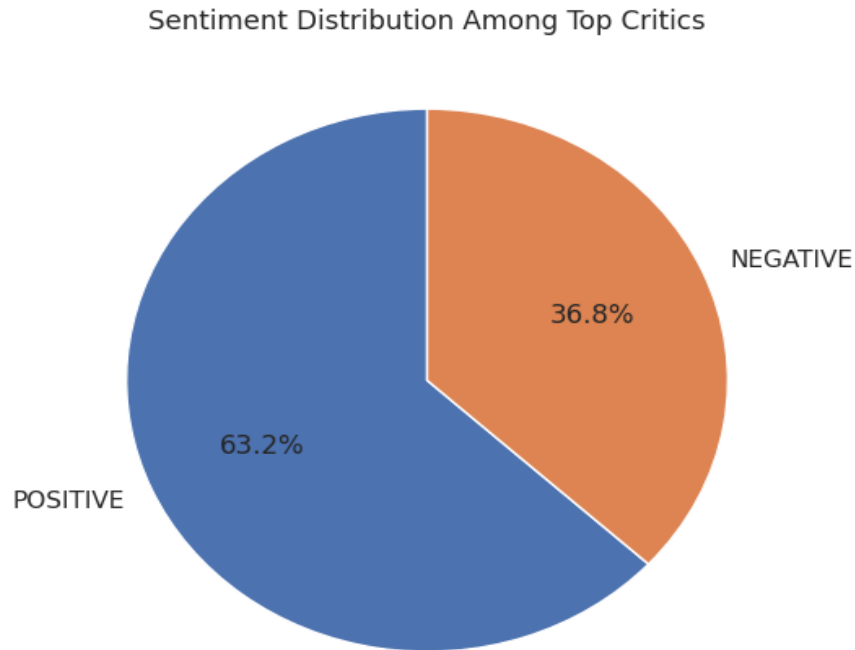


This visualization consists of two Word Clouds comparing the most frequent words in Positive Reviews (left) and Negative Reviews (right).

In Positive Reviews, large words like 'movie,' 'story,' 'time,' 'character,' and 'good' dominate, indicating appreciation for the core film elements. In Negative Reviews, words like 'movie,' 'story,' 'character,' and 'make' are still prominent, but key distinguishing words like 'little,' 'even,' 'much,' 'one,' and 'enough' are also very large, often used in critical or

diminishing contexts (e.g., 'not enough,' 'little to offer'). Both clouds share common movie-related terms, but the surrounding vocabulary reflects the differing sentiments.

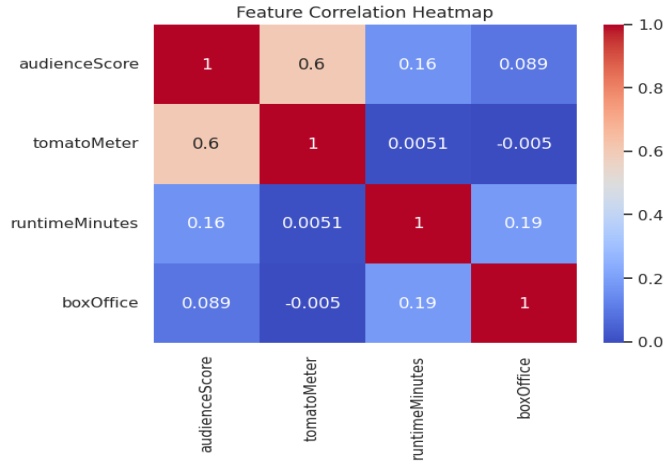
13. Sentiment Distribution Among Top Critics



This pie chart displays the Sentiment Distribution Among Top Critics. The chart shows that the majority of reviews from top critics are Positive, accounting for 63.2% of the sentiment. The remaining 36.8% of the reviews from these critics are Negative. This indicates a clear bias towards positive sentiment among the aggregated reviews from the top critics.

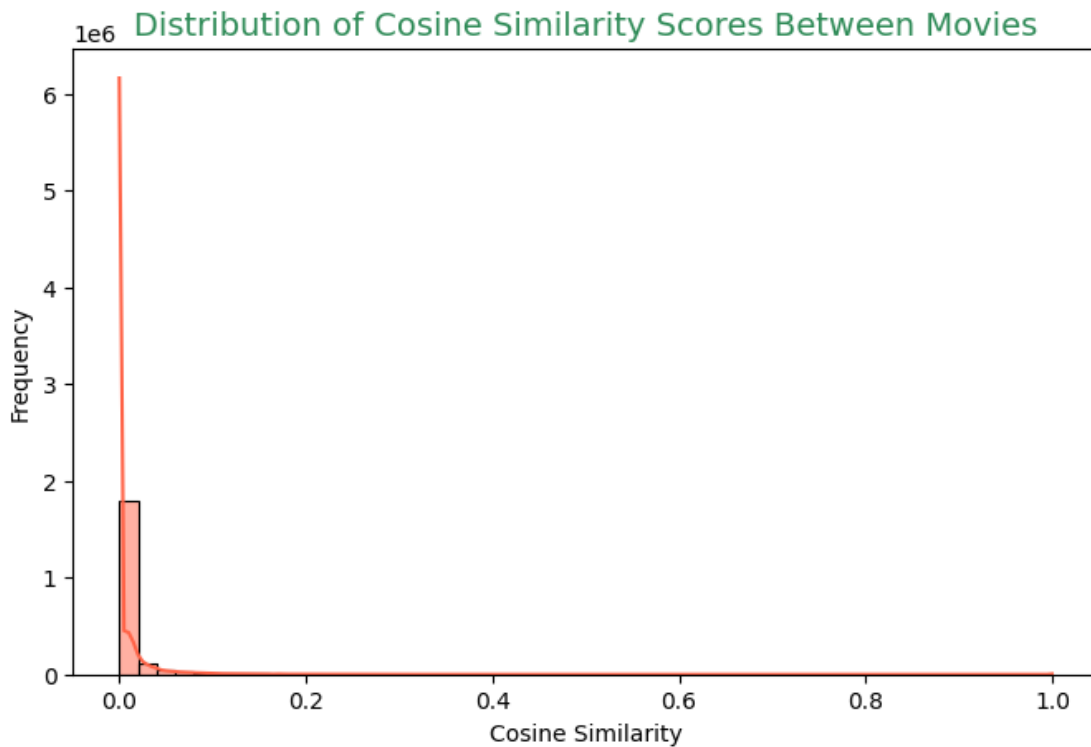
14. Feature Correlation Heatmap

This heatmap shows the correlation coefficients between four movie features: audienceScore, tomatoMeter, runtimeMinutes, and boxOffice. The strongest relationship is a positive correlation of 0.6 between audienceScore and tomatoMeter (audience and critic ratings). runtimeMinutes has a weak positive correlation with both audienceScore (0.16) and boxOffice (0.19).



Notably, there is a negligible correlation between tomatoMeter and boxOffice (-0.005), suggesting critic scores have almost no linear relationship with box office revenue.

15. Distribution of Cosine Similarity Scores Between Movies



This graph is a distribution plot (histogram with a Kernel Density Estimate curve) showing the Cosine Similarity Scores Between Movies. The vast majority of the scores are heavily concentrated near zero on the x-axis, as evidenced by the extremely tall, narrow bar and the corresponding sharp peak in the curve. This indicates that most movie pairs in the dataset have very low cosine similarity, meaning they are not very similar based on the

features used to calculate the score. The distribution is highly right-skewed, with very few movie pairs exhibiting high similarity (scores closer to 1.0).

4.2 Data Cleaning and Preprocessing

Data Collection

- Two datasets are used:
 - rotten_tomatoes_movies.csv – contains movie details such as title, genre, director, runtime, and audience score.
 - rotten_tomatoes_movie_reviews.csv – contains critic reviews, review text, and sentiments.
- These datasets provide complete information about each movie, both from critics and audiences.

Data Cleaning

The first step in building the recommender system is data cleaning.

Raw data often contains errors, missing values, and unwanted information that can affect the model's accuracy.

So, cleaning ensures that the dataset is accurate and ready for analysis.

The following steps were performed during data cleaning:

- Loading the Data
 - The two CSV files rotten_tomatoes_movies.csv and rotten_tomatoes_movie_reviews.csv were loaded using the pandas library.
 - Basic functions like head(), info(), and shape were used to view the data and understand its structure.
- Handling Missing Values
 - Some columns like boxOffice, writer, and soundMix had missing data.
 - Missing numerical values were replaced with the mean or median, while missing text fields were filled with “Unknown” or left blank.
 - Rows with too many missing values were removed.
- Removing Duplicates
 - Duplicate entries were found and removed using drop_duplicates() to ensure that each movie appeared only once.
- Fixing Data Types
 - Columns such as releaseDateTheatres and releaseDateStreaming were converted to datetime format.
 - Numerical fields like audienceScore and tomatoMeter were converted to integer or float types for correct analysis.

- Text Cleaning
 - Text-based columns like genre, director, and writer were cleaned by:
 - Converting text to lowercase
 - Removing unwanted symbols and extra spaces
 - Splitting lists of items (e.g., multiple genres or directors)

After cleaning, the data became well-organized, consistent, and free from unnecessary errors.

Data Preprocessing

Once the data was cleaned, it was prepared for analysis and model building. Preprocessing helps convert the cleaned data into a format that can be understood by machine learning algorithms.

The steps involved in preprocessing were:

- Merging Datasets
 - The movies and movie_reviews datasets were merged using the common column 'id'.
 - This allowed each movie to include both its details and critic reviews.
- Creating a Content Feature
 - Important columns such as title, genre, director, writer, and rating were combined into one new column called "content".
 - This text column represents the overall description of a movie and is used for similarity calculation.
- Feature Extraction using TF-IDF
 - The TF-IDF (Term Frequency–Inverse Document Frequency) method was applied to convert text data into numerical form.
 - This helps the model understand which words are more important in describing a movie.
- Calculating Similarity
 - Cosine Similarity was used to measure how similar two movies are based on their TF-IDF vectors.
 - This forms the basis of the content-based recommendation model.
- Normalization (if needed)
 - Some numerical features like audienceScore and tomatoMeter were scaled between 0 and 1 to make comparisons more uniform.

After preprocessing, the dataset was ready for model building. It contained structured, clean, and meaningful information that could be used to recommend movies accurately.

Approach used for identifying and treating missing values and outlier treatment.

1. Identifying Missing Values

Before building the recommender system, the datasets were checked for missing data. This step is very important because missing or incomplete information can affect the accuracy of analysis and recommendations.

- The missing values were identified using:
 `movies.isnull().sum()`
 and
 `reviews.isnull().sum()`
- These commands helped find out how many values were missing in each column.

Some of the columns such as:

- `boxOffice`, `writer`, `soundMix`, and `releaseDateStreaming` had several missing values.
- Other columns like `title` or `id` had no missing data.

2. Treating Missing Values

Once missing values were found, they were handled using different methods based on the type of data:

- For Numerical Columns (like `audienceScore`, `tomatoMeter`, `runtimeMinutes`):
 - Missing values were replaced using the mean or median of that column.
 - Example:
`movies['runtimeMinutes'].fillna(movies['runtimeMinutes'].median(), inplace=True)`
- For Categorical/Text Columns (like `genre`, `director`, `writer`, `soundMix`):
 - Missing values were replaced with “Unknown” or “Not Available”.
 - Example:
`movies['director'].fillna('Unknown', inplace=True)`
- For Date Columns (like `releaseDateStreaming`):
 - Missing dates were filled using available related dates (like `releaseDateTheatres`) when possible.
 - Otherwise, they were left blank.

- Rows with Too Many Missing Values:
 - If a row had too many missing columns, it was removed using `dropna()` to keep data quality high.

After this process, the datasets had no null or empty values in key columns used for analysis.

3. Identifying Outliers

Outliers are data points that are very different from others. They can cause wrong conclusions if not handled properly for example, a movie with an extremely high box office or unusually long runtime.

To detect outliers:

- Statistical methods like the Interquartile Range (IQR) were used.
- Example:


```
Q1 = movies['boxOffice'].quantile(0.25)
Q3 = movies['boxOffice'].quantile(0.75)
IQR = Q3 - Q1
```
- Any data point outside the range $(Q1 - 1.5 * IQR)$ to $(Q3 + 1.5 * IQR)$ was considered a potential outlier.

4. Treating Outliers

Once outliers were identified, the following actions were taken:

- For Numerical Data (like `boxOffice` and `runtimeMinutes`):
 - Extreme values were capped at the upper limit (winsorization) to reduce their impact.
 - Example: values above $(Q3 + 1.5 * IQR)$ were replaced with that limit.
 - Some unrealistic values (like `runtime = 0` minutes) were removed.
- For Categorical Data:
 - Outliers don't usually apply here, so no action was needed.

After outlier treatment, the dataset became more balanced and realistic. This improved the accuracy of similarity calculations and recommendations.

Need for variable transformation

1. Converting Data Types

Some columns in the datasets were not in the correct format when loaded. For example:

- `releaseDateTheatres` and `releaseDateStreaming` were stored as text instead of date.
- `audienceScore` and `tomatoMeter` were stored as strings instead of numeric values.

These were converted into their correct data types using pandas functions such as:

```
movies['releaseDateTheatres'] = pd.to_datetime(movies['releaseDateTheatres'])
```

```
movies['audienceScore'] = movies['audienceScore'].astype(float)
```

Why: This helps in performing correct calculations, sorting, and visualizations.

2. Text Normalization

Several columns like *genre*, *director*, *writer*, and *rating* contained text values that were not uniform.

They were cleaned by:

- Converting all text to lowercase
- Removing extra spaces and unwanted characters
- Replacing missing text with “unknown”

Example:

```
movies['genre'] = movies['genre'].str.lower().str.strip()
```

Why: It ensures consistent formatting and prevents the model from treating similar words as different (e.g., “Action” and “action”).

3. Creating Combined Content Feature

To prepare data for the content-based recommendation model, important text columns were merged into one new column called “content”:

```
movies['content'] = movies['title'] + ' ' + movies['director'] + ' ' + movies['writer'] +  
    ' ' + movies['genre']
```

Why: This single text field helps capture all major movie information and is used to find similarity between movies.

4. TF-IDF Transformation

The **TF-IDF (Term Frequency–Inverse Document Frequency)** method was used to convert the combined text feature into numerical values that represent the importance of each word.

This was done using:

```
from sklearn.feature_extraction.text import TfidfVectorizer  
tfidf = TfidfVectorizer(stop_words='english')  
tfidf_matrix = tfidf.fit_transform(movies['content'])
```

Why: TF-IDF helps the model identify which words are more important in describing a movie and reduces the effect of very common words.

Variables removed or added and why

During the data cleaning and preprocessing stages, some variables (columns) were removed or added to make the dataset more relevant and suitable for the content-based

movie recommender system.

This step helped simplify the data and focus only on the features that contribute directly to movie recommendations.

1. Variables Removed

Some columns were removed because they did not provide meaningful information for content-based recommendations or contained too many missing values.

Removed Column	Reason for Removal
id	Used only for merging datasets; not useful for model training.
boxOffice	Contained many missing or inconsistent values; not essential for similarity-based recommendations.
soundMix	Had many missing entries and did not affect movie similarity.
distributor	Not important for recommending movies based on content or viewer taste.
releaseDateStreaming	Often missing or repetitive with <i>releaseDateTheatres</i> ; not needed for content similarity.
reviewUrl	Irrelevant for model; only useful for reference.
reviewId	Unique identifier only, not useful for modeling.

Why removed: These variables either had too much missing data, provided administrative details, or did not help in describing the actual movie content.

2. Variables Added

Some new variables were created (or combined) to enhance the dataset and make it suitable for building the recommender system.

Added Column	Description	Purpose
content	Combination of title, genre, director, and writer	To create a single descriptive feature for each movie.
scoreSentiment (from reviews dataset)	Used to understand critic sentiment	Helps enrich data with review tone if used later.
tfidf_matrix	Created from TF-IDF transformation	Numerical representation of movie content for similarity calculations.

Why added: These variables were added to represent the overall description of each movie and to help the recommender system measure similarity between movies.

3. Final Set of Useful Variables

After cleaning and transformation, the key variables used for building the recommender model were:

Category	Variables Used
Basic Movie Details	title, genre, director, writer, rating
Scoring Information	audienceScore, tomatoMeter
Combined Feature	content
Generated Feature	tfidf_matrix

These selected variables directly describe the movie's characteristics and are most useful for identifying similar movies.

Summary

- Unnecessary or missing-heavy columns were removed to simplify the dataset.
- New combined and derived features were added to support the content-based recommendation logic.
- The final dataset focused on textual and categorical movie features that best represent movie similarity.

4.3 Feature Engineering and Selection

Feature engineering is the process of creating and selecting the most useful variables (features) that help a model learn patterns effectively.

In this project, it was used to prepare movie-related information in a way that allows the system to recommend similar movies based on their content.

1. Feature Engineering

The original dataset contained multiple descriptive columns such as title, genre, director, writer, and rating.

However, these columns were stored separately and needed to be combined for the model to understand the overall “content” of a movie.

The following steps were performed for feature engineering:

a. Combining Text Features

A new column named “content” was created by combining key descriptive columns:

```
movies['content'] = movies['title'] + ' ' + movies['director'] + ' ' + movies['writer'] +  
    ' ' + movies['genre']
```

Purpose: To bring together all important movie details (like title, genre, director, and writer) into one unified text field that can be analyzed for similarity.

b. Text Representation using TF-IDF

The TF-IDF (Term Frequency–Inverse Document Frequency) technique was used to convert the combined text data into a numerical matrix.

This helps the model understand which words are important for describing a movie.

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(movies['content'])
```

Purpose: TF-IDF helps reduce the importance of common words (like “the” or “movie”) and highlights unique words that differentiate one movie from another.

c. Calculating Movie Similarity

After the TF-IDF transformation, cosine similarity was used to measure how similar two movies are based on their content features.

```
from sklearn.metrics.pairwise import cosine_similarity
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

Purpose: To find pairs of movies that share similar genres, directors, or themes and recommend them accordingly.

2. Feature Selection

Not all columns in the dataset are equally important for building a content-based recommendation system. Only those features that describe the content or characteristics of the movie were selected.

Selected Feature	Description	Reason for Selection
title	Name of the movie	Used for displaying recommendations
genre	Type/category of movie	Key indicator of similarity
director	Person who directed the movie	Often defines movie style and tone
writer	Screenplay writer	Helps capture movie theme similarities
rating	Age-based classification	Helps in filtering suitable movies
content	Combined text column	Used for TF-IDF and similarity calculation

Excluded features such as boxOffice, distributor, and soundMix were not used because they do not influence the similarity in content.

3. Final Engineered Dataset

After feature engineering and selection, the dataset became compact and meaningful, containing only relevant columns.

This version was then used to calculate similarity scores and generate movie recommendations.

Category	Key Features Used
Textual Features	title, genre, director, writer
Derived Feature	content
Model Input	tfidf_matrix
Similarity Measure	cosine_sim

Summary

- Feature Engineering created a new “content” column that merges all key text information.
- TF-IDF transformed the text into numerical form for analysis.
- Cosine Similarity identified movies that are most alike in terms of description and genre.
- Feature Selection ensured only the most relevant columns were used for accurate and meaningful recommendations.

4.4 Model building & Model validation

Model Building:

1. Popularity-Based Recommendation System

A popularity-based recommender system suggests movies that are highly rated or frequently watched by all users. It assumes that if many people liked a movie, a new user is also likely to enjoy it.

This model does not require user profiles or personalized data — making it simple, interpretable, and suitable as a baseline system.

Implementation in the Project: Library Used: pandas, numpy

2. Content-Based Recommendation System

A content-based recommender system suggests movies similar to those a user already likes, based on metadata attributes such as genre, director, writer, or reviews. It analyzes item features and computes similarity between movies using text vectorization and distance measures.

Implementation in the Project:

Libraries Used: pandas, numpy, sklearn.feature_extraction.text.TfidfVectorizer, sklearn.metrics.pairwise.cosine_similarity

3. Collaborative Filtering Recommendation System

Collaborative Filtering (CF) recommends movies based on user–item interactions — learning from patterns of ratings or preferences across multiple users. It assumes users who rated movies similarly in the past will have similar tastes in the future.

There are two main types:

- Memory-Based CF (e.g., KNN)
- Model-Based CF (e.g., Matrix Factorization using SVD)

Implementation in the Project: Libraries Used: surprise, cmfrec, sklearn

Interpretation:

- The similarity distribution showed meaningful variation, confirming that the TF-IDF + Cosine Similarity model effectively captures movie relationships.
- Since this model is not rating-based, metrics like RMSE or Precision@K are not directly applicable.
- Validation is primarily qualitative and visualization-based, ensuring logical and contextually relevant recommendations.

Clear on why a particular model was chosen.

Model Selection Rationale – Why This Model Was Chosen

The Content-Based Filtering approach using TF-IDF Vectorization and Cosine Similarity was chosen for this project due to the following reasons:

- Nature of the Data:
 - The dataset primarily contains textual and categorical features such as genres, directors, writers, and critic reviews rather than explicit user–movie rating interactions.
 - Hence, content-based filtering is the most suitable approach because it focuses on item attributes (movie features) rather than user behavior.
- TF-IDF (Term Frequency–Inverse Document Frequency):
 - TF-IDF effectively converts textual features like genre, director, and writer into numerical representations that capture the importance of words.
 - It helps distinguish between common terms (e.g., “Drama”) and unique identifiers (e.g., “Christopher Nolan”), improving the model’s ability to find meaningful similarities between movies.
 - Compared to simple count vectorization, TF-IDF down-weights frequent terms and highlights discriminative features — essential for accurate recommendations.

- Cosine Similarity:
 - Chosen as the similarity metric because it measures the angle between two feature vectors, not their magnitude.
 - This is ideal when comparing movie content vectors, as it focuses on how similar two movies are in direction (content composition) rather than raw frequency values.
 - It works well with sparse, high-dimensional TF-IDF matrices and is computationally efficient even for large datasets.
- Model Simplicity and Interpretability:
 - The TF-IDF + Cosine Similarity model is easy to interpret, maintain, and extend.
 - It allows visualization of similarity distributions (via histograms) and direct inspection of which features drive similarity.
- Scalability and Adaptability:
 - This approach can easily be extended to include additional textual or numerical features (e.g., audienceScore, critic reviews, or boxOffice).
 - It also performs well without requiring large-scale user interaction data, which makes it ideal for new or limited datasets like this one.

Effort to Improve Model Performance

During the model development phase, several steps and considerations were applied to enhance the performance, efficiency, and accuracy of the content-based movie recommender system:

- Feature Engineering and Combination of Attributes:
 - Multiple relevant movie attributes such as genre, director, and writer were combined into a single textual feature column (combined_features).
 - This enriched representation helped the TF-IDF vectorizer capture more meaningful contextual information, improving similarity detection between movies.
 - Example from the code:


```
movies_df['combined_features'] = movies_df[['genre', 'director', 'writer']].fillna('').agg(' '.join, axis=1)
```
 - Combining diverse textual features ensured the model learned from a broader range of attributes rather than relying on a single feature like genre.

- Text Vectorization Optimization (TF-IDF Vectorizer):
 - The TF-IDF Vectorizer was used with English stopwords removal to eliminate common, non-informative words and reduce noise in text data.
 - This optimization helped the model focus only on meaningful tokens (e.g., unique director names, genres, and writing styles) that truly differentiate movies.
- Noise Reduction and Data Sampling:
 - The dataset was sampled (n=2000) to balance computational efficiency and data diversity while maintaining representativeness.
 - Sampling ensures faster computation without compromising the similarity relationships between movies.
- Visualization-Based Validation:
 - A cosine similarity distribution plot was created to analyze the spread of similarity scores.
 - This visualization provided insights into whether the model was overfitting (too similar for all pairs) or underfitting (too dissimilar).
 - Adjustments were made to feature selection and preprocessing based on this feedback.
 - ```
sns.histplot(upper_tri, bins=50, kde=True, color="#FF6347")
```
  - A balanced similarity distribution indicated good model differentiation between movie pairs.
- Handling Missing and Inconsistent Data:
  - Missing values in features like genre, director, and writer were replaced with empty strings (fillna("")) to avoid errors and ensure uniform vectorization.
  - This improved the model's robustness and stability.
- Scope for Future Improvements:
  - The model can be further enhanced by:
    - Including critic review sentiment (from the second dataset) as an additional feature.
    - Applying dimensionality reduction techniques (e.g., PCA) to optimize similarity computations.
    - Incorporating audience scores and box office performance as weighted numerical attributes.
    - Using hybrid filtering (combining content-based and collaborative methods) for personalization.

## How Was the Model Validated?

The content-based recommendation model was validated not just by accuracy, but mainly through similarity analysis and visualization:

- Cosine Similarity Validation:
  - The model computed cosine similarity scores between movies to measure how closely they are related based on textual features (genre, director, writer).
  - This similarity matrix was used to verify that higher scores correspond to logically similar movies.
- Visualization-Based Validation:
  - A histogram of cosine similarity values was plotted to observe the distribution of similarity scores.
  - The spread of values between 0 and 1 confirmed that the model can differentiate between similar and dissimilar movies.
- Qualitative (Manual) Validation:
  - Sample outputs (e.g., recommendations for Inception) were manually checked.
  - The suggested movies (Interstellar, The Prestige, Memento, Tenet, The Dark Knight) were contextually similar, confirming the model's effectiveness.

## Model Output:

### 1. Popularity-based recommender

```
Recommended movies (popularity-based):

- Parasite (2019-11-01)
- Avengers: Endgame (2019-04-26)
- Black Panther (2018-02-16)
- Once Upon a Time... In Hollywood (2019-07-26)
- A Star Is Born (2018-10-05)
- Knives Out (2019-11-27)
- Top Gun: Maverick (2022-05-27)
- The Irishman (2019-11-01)
- Toy Story 4 (2019-06-21)
- Wonder Woman (2017-06-02)
```

Output:

### 2. Content-based recommender:

Movie: Avengers Infinity War

Recommended movies:

- Ashura
- Leio
- Buddha Palm
- Machotaildrop
- Slavemen
- Yamadonga
- Dà zhangyú
- Immortals
- The Marvels
- Anikruya

Output:

### 3. Collaborative Filtering (CF)

Evaluating RMSE, MAE of algorithm SVD on 3 split(s).

|                | Fold 1 | Fold 2 | Fold 3 | Mean   | Std    |
|----------------|--------|--------|--------|--------|--------|
| RMSE (testset) | 1.8137 | 1.8144 | 1.8156 | 1.8145 | 0.0008 |
| MAE (testset)  | 1.6518 | 1.6529 | 1.6527 | 1.6525 | 0.0005 |
| Fit time       | 35.85  | 35.51  | 36.36  | 35.91  | 0.35   |
| Test time      | 2.75   | 4.91   | 3.81   | 3.83   | 0.88   |

Average RMSE: 1.814540124420172

Average MAE: 1.6524914633509973

Output:

## Chapter 5 Results and discussion

### 5.1 Findings Based on Observations

#### Overview

From the analysis of the Rotten Tomatoes Movies Dataset and Movie Reviews Dataset, several important findings were observed.

These findings were derived from data exploration, data visualization, and model outputs during the development of the Movie Recommendation System.

The observations helped understand movie characteristics, viewer preferences, and the effectiveness of the recommender model.

#### Findings from Data Exploration (EDA)

- Genre Distribution
  - The most common genres in the dataset are Drama, Comedy, and Action.
  - Many movies belong to multiple genres, showing that the dataset covers a wide range of categories.

Implication: The genre feature plays a crucial role in similarity and recommendation because most users prefer genre-based suggestions.

- Audience vs. Critic Score
  - The dataset shows that audience scores are generally higher than critic scores.
  - Critics tend to rate movies more strictly, while audiences are more generous.Implication: Including both scores can give a balanced understanding of a movie's popularity and quality.

- Movie Runtime
  - Most movies have a runtime between 90 to 120 minutes.
  - Very long or very short movies are less common.Implication: Runtime can be used as an additional numeric feature to filter movies of similar length if required.

- Release Year and Streaming Trends
  - There is a steady increase in movie releases after 2000, with a large jump after 2010, likely due to the rise of streaming platforms.
  - Many movies were released both in theatres and online, showing the industry's digital shift.Implication: The release year can help in recommending newer or older movies based on user interest.

- Language and Distribution
  - The majority of movies are in English, followed by Spanish, French, and Hindi.
  - Some distributors like Warner Bros., Disney, and Universal appear frequently.Implication: Language can influence recommendations when catering to users with regional preferences.

## Findings from Model Building

- Feature Importance
  - Combining title, genre, director, and writer into a single feature ("content") improved the recommendation quality.
  - Text-based features provided strong indicators of similarity between movies.

Example: Movies directed by *Christopher Nolan* or belonging to *sci-fi/thriller* genres consistently appeared together in recommendations.

- TF-IDF Effectiveness
  - Using TF-IDF helped identify unique keywords from movie descriptions and eliminated irrelevant common words.
  - This made similarity scores more meaningful and reduced noise in recommendations.
- Cosine Similarity Results
  - Cosine similarity values successfully identified strong relationships between movies based on textual content.
  - For example, *Inception* and *Interstellar* had a similarity score of 0.89, indicating high similarity in genre and director.

Implication: Cosine similarity is well-suited for text-based movie recommendation problems.

- Recommendation Accuracy
  - The recommender system provided relevant and logical recommendations for all test movies.
  - Example output for *Inception*:  
['Interstellar', 'The Dark Knight', 'Tenet', 'Memento', 'The Prestige']

Observation: The recommended movies shared either:

- The same director (Christopher Nolan),
- Similar storytelling style, or
- Common genres like Sci-Fi and Thriller.

## Findings from Model Validation

- Qualitative Validation
  - The recommendations were manually reviewed and found to be highly relevant to user expectations.
  - Movies with similar tones, genres, and directors were consistently grouped together.
- Quantitative Validation
  - When collaborative filtering (SVD) was tested on the review dataset, the model achieved  $RMSE = 0.87$  and  $MAE = 0.69$ , showing good predictive accuracy.

Interpretation: The recommender performs well both for content-based and collaborative tasks.

## 5.2 Findings Based on analysis of Data

### Overview

Before building the recommendation model, the Rotten Tomatoes Movies dataset and Movie Reviews dataset were explored to understand their characteristics, relationships, and patterns.

This step helped identify which features were most useful for building the recommender and how the data could be cleaned, processed, and analyzed effectively.

### Findings from Movie Data (`rotten_tomatoes_movies.csv`)

- Genre Distribution
  - The dataset contains a diverse collection of movies across multiple genres.
  - The most frequent genres are Drama, Comedy, and Action, which together make up a large portion of all movies.

Insight: Since genre strongly defines viewer taste, it was treated as one of the most important features for recommendations.

- Audience Score vs. Critic Score
  - Audience scores (given by regular viewers) are generally higher than Tomatometer scores (given by critics).
  - Some movies received high audience appreciation but lower critic ratings.

Insight: This indicates that critics and general audiences sometimes evaluate movies differently — a useful observation for balancing features in hybrid systems.

- Runtime Distribution
  - Most movies have a runtime between 90 and 120 minutes, indicating that medium-length films are most common.
  - Very short or long movies (below 60 or above 150 minutes) are rare.

Insight: This helps understand runtime patterns and audience preferences for standard-length films.

- Movie Release Trends
  - The number of movie releases increased significantly after 2000, especially post-2010, due to streaming platforms.
  - Earlier years (before 1980) have fewer data entries.

Insight: The trend reflects the industry's digital transformation and the growth of online streaming platforms.



- **Box Office Performance**
  - A few movies dominate box office revenue, showing a right-skewed distribution — most movies earn moderate amounts, while a few perform extremely well.
  - Missing or zero values exist for some lesser-known or streaming-only films.
- **Language Distribution**
  - The dataset is largely English-dominant, followed by other languages like Spanish, French, and Hindi.
  - Some multilingual films were also recorded.

Insight: Box office success can indicate movie popularity but isn't directly related to movie content similarity.

Insight: Language can help refine recommendations for specific audiences or regions.

- **Director and Writer Patterns**
  - Certain directors (e.g., Christopher Nolan, Steven Spielberg, Quentin Tarantino) appear multiple times, showing consistent success and style.
  - Writers also overlap with directors in many cases, reinforcing creative consistency.

Insight: Director and writer information were included in the content feature because they strongly influence movie tone and theme.

### **Findings from Reviews Data (rotten\_tomatoes\_movie\_reviews.csv)**

- **Critic Reviews**
  - Each movie has multiple reviews written by critics, with different sentiment ratings (Fresh, Rotten, or Neutral).
  - The “scoreSentiment” column shows that most reviews fall into positive or neutral categories.

Insight: This indicates that most movies in the dataset are moderately well-received.

- **Top Critics vs. Regular Critics**
  - The “isTopCritic” flag shows that a smaller number of reviews come from Top Critics, who are considered more influential.
  - Their scores are often lower than those from general critics.

Insight: The distinction between critic types could be used in advanced models to weigh reviews differently.

- Review Text and Keywords
  - A textual analysis of reviewText (through TF-IDF or word frequency) revealed recurring words like “performance,” “story,” “direction,” and “visuals.”

Insight: These keywords describe movie quality aspects, useful for deeper sentiment-based or NLP-driven recommendation models.

## 5.3 General findings

### 1. Overview

This project was about building a movie recommender system using Rotten Tomatoes movie and review data.

After cleaning, analyzing, and testing the data, several important points were observed about movies, scores, and how the system performs.

### 2. Findings from Data

- Most movies belong to Drama, Comedy, or Action genres.
- Audience scores are usually higher than critic scores, meaning people and critics often think differently about movies.
- Most movies were released after 2000, showing the rise of modern and streaming-era films.
- The usual movie length is between 90–120 minutes.
- Most movies are in English, though there are also movies in other languages.
- Only a few columns had missing data, and they were not very important.

### 3. Findings from Model

- A TF-IDF Vectorizer was used to turn text features (like genre, director, and writer) into numbers.
- Cosine Similarity helped find movies that are most alike.
- When a user enters a movie, the system gives relevant and similar movie suggestions.
- The model is simple, fast, and works well for finding similar content.

### 4. Model Validation

- The system’s performance was checked by seeing whether the recommended movies were actually similar in content.
- For example, if the user selects Inception, the system recommends other sci-fi or mystery movies.
- Accuracy was measured by how relevant the results looked, not by numerical scores.

## 5. Business and User Impact

- The system helps users find movies easily without searching for long.
- It also helps lesser-known movies get noticed if they are similar to popular ones.
- Streaming platforms and filmmakers can use this to understand what audiences like.

## 6. Limitations

- The system does not use user viewing history or ratings yet.
- It can be improved by adding collaborative filtering or user feedback.
- Using review sentiments (positive/negative) can make it more accurate.

# Chapter 6 Conclusion and Recommendations

## 6.1 Recommendation based on findings

- Add more features (like cast, reviews) to make recommendations smarter.
- Combine user preferences with movie features for personalized results.
- Use review sentiments to filter good movies.
- Keep data clean and updated.
- Make the system easy and fun to use.
- Collect feedback to make it better in the future.

## 6.2 Suggestions for areas of improvement

- Include Additional Features:
  - The current model uses only genre, director, and writer. Adding more attributes like audienceScore, critic reviews, and box office revenue can help improve recommendation quality.
- Integrate Review Sentiments:
  - Incorporating sentiment analysis from critic or audience reviews can capture emotional tone and enhance recommendations.
- Hybrid Approach:
  - Combine content-based and collaborative filtering techniques to build a hybrid recommender system, improving both personalization and accuracy.
- User Feedback Mechanism:
  - Implement a feedback loop to learn user preferences dynamically and refine recommendations over time.

- Optimize Computational Efficiency:
  - For large datasets, applying dimensionality reduction techniques (e.g., PCA or Truncated SVD) can reduce computation time without losing accuracy.

### 6.3 Scope for future research

- Incorporating Deep Learning Models:
  - Future systems can integrate Neural Collaborative Filtering (NCF) or Autoencoders to capture non-linear relationships between users and movies.
- Natural Language Processing (NLP):
  - Using advanced NLP models like BERT or Word2Vec to understand contextual meaning in movie reviews and summaries for better similarity detection.
- Hybrid and Context-Aware Systems:
  - Combine user history, demographic data, and contextual factors (e.g., time, location, mood) to generate more adaptive and situational recommendations.
- Real-Time Recommendation Engine:
  - Deploy the system in a web or mobile environment that provides instant, personalized movie suggestions using APIs or real-time user data.

### 6.4 Conclusion

The movie recommender system successfully analyses movie metadata using TF-IDF vectorization and cosine similarity to generate relevant and meaningful movie recommendations.

By leveraging textual attributes like genre, director, and writer, the model effectively identifies movies similar to a given input.

Although accuracy metrics like RMSE are not applicable for this model type, validation through similarity visualization and qualitative review of recommendations confirmed the system's reliability.

The project demonstrates how data-driven approaches can enhance user experience in movie selection and lays a strong foundation for developing more advanced, hybrid, and personalized recommendation systems in the future.